

QUESTION-8

Single linked list using deletion at beginning, end, at any position

main.c	Output
<pre>1 #include <stdio.h> 2 #include <stdlib.h> 3 struct node { 4 int data; 5 struct node *next; 6 }; 7 struct node *head = NULL; 8 void insertEnd(int value) { 9 struct node *newNode, *temp; 10 newNode = (struct node *)malloc(sizeof 11 (struct node)); 12 newNode->data = value; 13 newNode->next = NULL; 14 if (head == NULL) { 15 head = newNode; 16 return; 17 } 18 temp = head; 19 while (temp->next != NULL) 20 temp = temp->next; 21 temp->next = newNode; 22 } 23 void deleteBegin() {</pre>	<pre>Original List: Linked List: 10 -> 20 -> 30 -> 40 -> 50 -> NULL After Deletion at Beginning: Linked List: 20 -> 30 -> 40 -> 50 -> NULL After Deletion at End: Linked List: 20 -> 30 -> 40 -> NULL After Deletion at Position 2: Linked List: 20 -> 40 -> NULL === Code Execution Successful ===</pre>

main.c	Output
<pre>24 if (head == NULL) { 25 printf("List is Empty\n"); 26 return; 27 } 28 temp = head; 29 head = head->next; 30 free(temp); 31 } 32 void deleteEnd() { 33 struct node *temp, *prev; 34 if (head == NULL) { 35 printf("List is Empty\n"); 36 return; 37 } 38 if (head->next == NULL) { 39 free(head); 40 head = NULL; 41 return; 42 } 43 temp = head; 44 while (temp->next != NULL) { 45 prev = temp; 46 temp = temp->next; 47 }</pre>	<pre>Original List: Linked List: 10 -> 20 -> 30 -> 40 -> 50 -> NULL After Deletion at Beginning: Linked List: 20 -> 30 -> 40 -> 50 -> NULL After Deletion at End: Linked List: 20 -> 30 -> 40 -> NULL After Deletion at Position 2: Linked List: 20 -> 40 -> NULL === Code Execution Successful ===</pre>

main.c	Output
<pre>51 void deletePosition(int pos) { 52 struct node *temp, *ptr; 53 int i; 54 if (head == NULL) { 55 printf("List is Empty\n"); 56 return; 57 } 58 if (pos == 1) { 59 temp = head; 60 head = head->next; 61 free(temp); 62 return; 63 } 64 temp = head; 65 for (i = 1; i < pos - 1 && temp != NULL; i++) 66 temp = temp->next; 67 if (temp == NULL temp->next == NULL) { 68 printf("Invalid Position\n"); 69 return; 70 } 71 ptr = temp->next; 72 temp->next = ptr->next; 73 free(ptr);</pre>	<pre>Original List: Linked List: 10 -> 20 -> 30 -> 40 -> 50 -> NULL After Deletion at Beginning: Linked List: 20 -> 30 -> 40 -> 50 -> NULL After Deletion at End: Linked List: 20 -> 30 -> 40 -> NULL After Deletion at Position 2: Linked List: 20 -> 40 -> NULL === Code Execution Successful ===</pre>

main.c		Output
<pre>80 temp = temp->next; 81 } 82 printf("NULL\n"); 83 } 84 int main() { 85 insertEnd(10); 86 insertEnd(20); 87 insertEnd(30); 88 insertEnd(40); 89 insertEnd(50); 90 printf("Original List:\n"); 91 display(); 92 deleteBegin(); 93 printf("After Deletion at Beginning:\n"); 94 display(); 95 deleteEnd(); 96 printf("After Deletion at End:\n"); 97 display(); 98 deletePosition(2); 99 printf("After Deletion at Position 2:\n"); 100 display(); 101 return 0; 102 }</pre>	<pre>Original List: Linked List: 10 -> 20 -> 30 -> 40 -> 50 -> NULL After Deletion at Beginning: Linked List: 20 -> 30 -> 40 -> 50 -> NULL After Deletion at End: Linked List: 20 -> 30 -> 40 -> NULL After Deletion at Position 2: Linked List: 20 -> 40 -> NULL === Code Execution Successful ===</pre>	